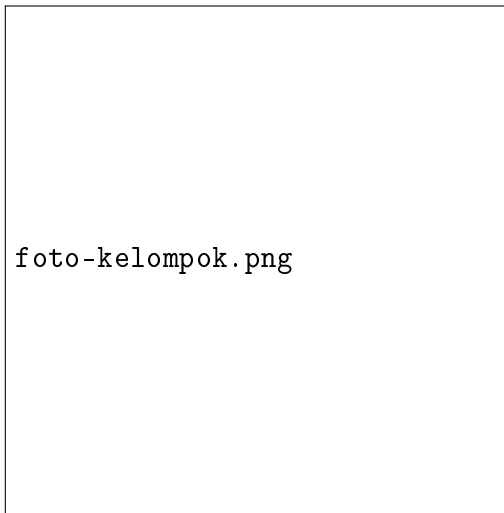

Laporan Tugas Besar 2

IF2211 Strategi Algoritma

PEMANFAATAN ALGORITMA BFS DAN DFS DALAM MEKANISME PENELUSURAN CSS PADA POHON *DOCUMENT OBJECT MODEL*

Semester II Tahun 2025/2026



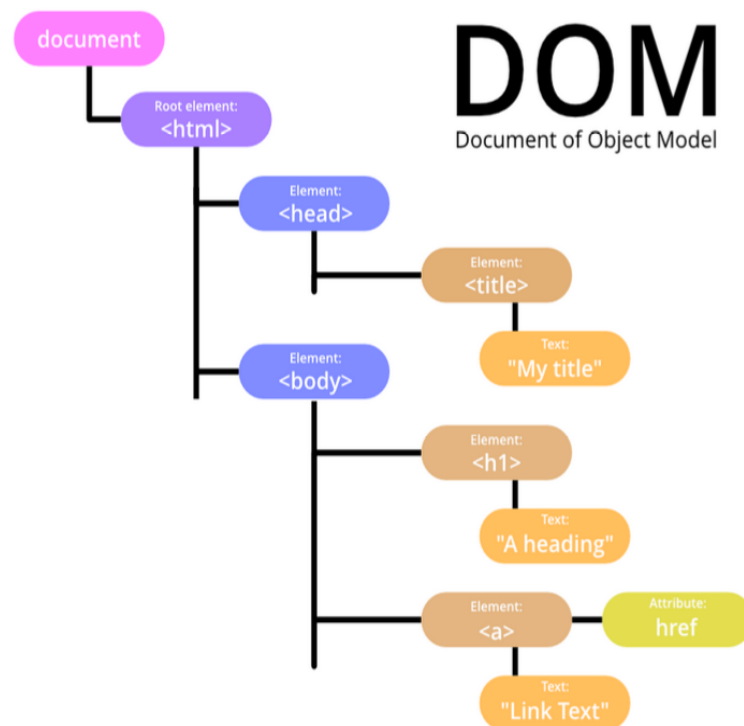
Muhammad Fatih Irkham Mauludi	(13524004)
Josh Reinhart Zidik	(13524048)
Jingglang Galih Rinenggan	(13524095)

LABORATORIUM ILMU DAN REKAYASA KOMPUTASI
PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

1	Deskripsi Tugas	3
2	Landasan Teori	6
2.1	Graph Traversal	6
2.2	Lowest Common Ancestor	14
2.3	Binary Lifting	15
2.4	HTML dan DOM Tree	19
2.5	CSS Selector	21
2.6	Client-Server Architecture	23
2.7	Parallel Computing	24
2.8	Virtualisasi dan Kontainerisasi	26
3	Aplikasi DFS dan BFS	29
3.1	Perancangan DFS dan BFS	29
3.2	Perancangan LCA	32
4	Implementasi dan Pengujian	34
4.1	Implementasi	34
4.2	Pengujian	39
5	Kesimpulan dan Saran	40
5.1	Kesimpulan	40
5.2	Saran	40

1 Deskripsi Tugas



Gambar 1: Struktur Pohon DOM (*Document Object Model*)

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk tree yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram.

Struktur sebuah file HTML pada umumnya meliputi elemen root `<html>` yang berisi dua bagian utama yaitu `<head>` dan `<body>`. Bagian `<head>` berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan `<body>` berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri

dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1. HTML Tags
2. CSS Selector
3. HTML parsing

Pada tugas ini, terdapat spesifikasi wajib untuk membuat aplikasi traversal pohon HTML (DOM Tree) yang menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) untuk melakukan pencarian elemen berdasarkan CSS selector pada struktur DOM dengan detail sebagai berikut,

- Buatlah sebuah aplikasi berbasis web yang terdiri dari frontend bebas dan backend dalam bahasa **Go**, **C#**, **Rust**, atau **Typescript** yang mengimplementasikan algoritma **BFS** dan **DFS** untuk melakukan penelusuran CSS pada pohon DOM.
- Aplikasi dapat melakukan scraping HTML dari sebuah website melalui URL yang diberikan oleh pengguna
- Secara umum, berikut adalah input yang diharapkan,
 1. URL website atau memasuki teks HTML sendiri
 2. Pilihan algoritma traversal (BFS / DFS)
 3. CSS selector
 4. Jumlah hasil yang ingin ditampilkan:
 5. Top n kemunculan
 6. Semua kemunculan
- HTML yang diperoleh kemudian diparsing menjadi struktur data pohon (DOM Tree).
- Output yang diharapkan adalah sebagai berikut:
 1. Aplikasi dapat menampilkan visualisasi struktur pohon DOM, disertai dengan keterangan kedalaman maksimum tree

2. Aplikasi dapat menampilkan highlight pada jalur yang di-traversal oleh algoritma penelusuran yang dipilih untuk menandakan elemen-elemen yang terpengaruh.
3. Aplikasi dapat menampilkan waktu pencarian serta banyak node yang dikunjungi.
4. Setelah melakukan penelusuran, aplikasi menyimpan sebuah traversal log yang memiliki informasi tahapan penelusuran yang dilakukan.

Selain itu terdapat spesifikasi bonus tugas besar (opsional) dengan poin tambahan tertentu yang dapat dipilih dengan detail sebagai berikut,

- (4 poin) [Video] Membuat video tentang aplikasi BFS dan DFS pada penelusuran pohon DOM kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan menampilkan wajah dari setiap anggota kelompok.
- (5 poin) [Deploy] Aplikasi web yang telah dibuat perlu di deploy menggunakan Microsoft Azure Virtual Machine (VM).
- (2 Poin) [Docker] Aplikasi dijalankan menggunakan Docker baik untuk frontend maupun backend.
- (6 Poin) [Animasi] Menampilkan animasi penelusuran pohon DOM untuk segala pencarian.
- (3 Poin) [Multithreading] Mengimplementasikan multithreading pada algoritma BFS dan/atau DFS untuk melakukan traversal pohon DOM secara paralel. Proses traversal dibagi ke beberapa thread/worker sehingga beberapa node dapat diproses secara bersamaan.
- (3 Poin) [LCA Binary Lifting] Mengimplementasikan algoritma Lowest Common Ancestor (LCA) dengan teknik Binary Lifting pada pohon DOM yang dihasilkan.

2 Landasan Teori

2.1 Graph Traversal

Graph Traversal atau traversal graf adalah operasi yang dapat dilakukan pada sebuah graf dengan mengunjungi simpul-simpul yang ada secara sistematis. Terdapat berbagai jenis traversal graf, misalnya pencarian melebar (*breadth first search*/BFS) dan pencarian mendalam (*depth first search*/DFS). Untuk melakukan traversal graf, terdapat asumsi bahwa semua simpul yang ingin dikunjungi harus saling terhubung (berada pada satu *connected component* yang sama) [1].

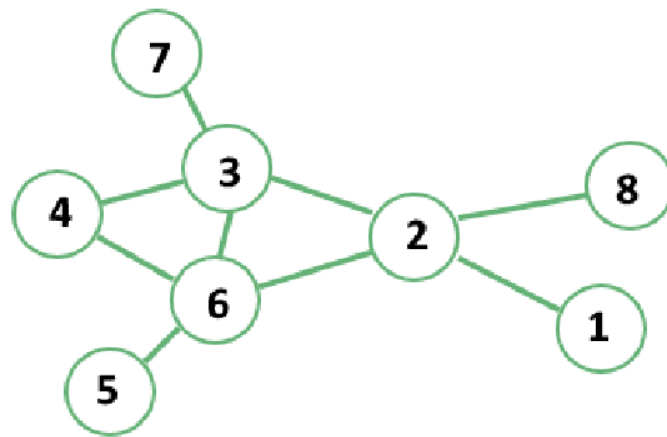
Setiap kali mengunjungi simpul, algoritma dapat melakukan suboperasi seperti mencetak informasi di dalam simpul, memanipulasi nilainya, dan lain-lain. Karena graf merupakan representasi suatu persoalan, maka melakukan traversal di dalam graf dapat diartikan sebagai melakukan pencarian solusi persoalan yang direpresentasikan dengan graf tersebut [1].

Algoritma pencarian pada graf berbasis solusi dapat diklasifikasikan sebagai [1]:

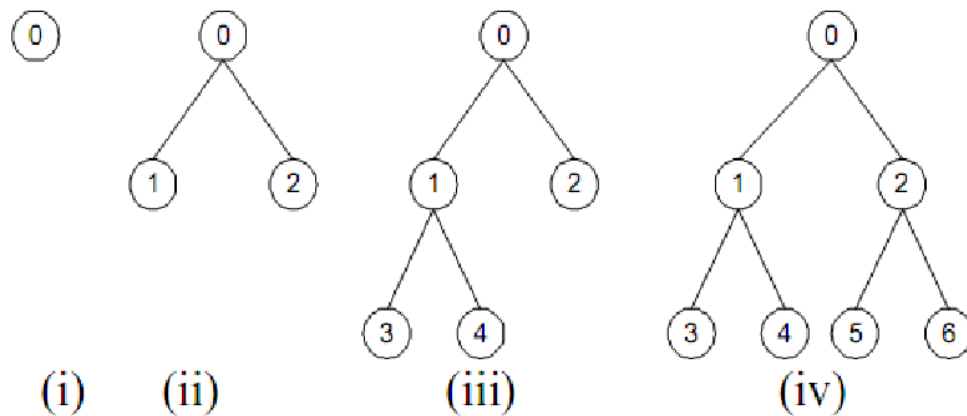
1. **Tanpa informasi** (*uninformed/blind search*)
 - Tidak ada informasi tambahan yang disediakan tentang seberapa dekat suatu simpul dengan tujuan
 - Satu-satunya informasi hanya dari graf itu sendiri
 - Contoh algoritma: DFS, BFS, Depth Limited Search, Iterative Deepening Search, Uniform Cost Search
2. **Dengan informasi** (*informed search*)
 - Pencarian berbasis heuristik
 - Menggunakan informasi tambahan berupa heuristic function untuk memperkirakan seberapa dekat suatu simpul ke tujuan, atau dengan kata lain simpul mana yang “lebih menjanjikan” ke simpul tujuan.
 - Contoh algoritma: Greedy Best First Search, A*

Representasi graf dalam pencarian solusi dapat dibagi menjadi dua pendekatan [2]:

1. **Graf statis**: graf yang sudah tersedia sebelum proses pencarian dilakukan dan langsung direpresentasikan sebagai struktur data
2. **Graf dinamis**: graf tidak tersedia sebelum pencarian dan dibangun secara bertahap selama pencarian solusi



Gambar 2: Contoh graf statis



Gambar 3: Contoh graf dinamis

Terkait pencarian solusi dengan traversal pada graph dinamis perlu memperhatikan hal-hal berikut [2]:

- Persoalan diselesaikan dengan melakukan pencarian solusi sambil membangun graf (umumnya bentuk khusus pohon) secara bertahap
- Pencarian solusi ekuivalen dengan pembentukan pohon dinamis
- Simpul dibuat lalu diperiksa apakah merupakan simpul solusi. Jika simpul merupakan solusi maka pencarian selesai (jika hanya satu solusi ditemukan) atau dilanjutkan mencari solusi lain (semua solusi ditemukan)
- Representasi pohon dinamis:
 1. Dinamakan sebagai **pohon ruang status** (*state space tree*)
 2. **Simpul** menyatakan status persoalan (*problem state*)

3. **Akar:** *initial state*
 4. **Daun:** *solution/goal state*
 5. **Cabang:** *operator*/langkah dalam persoalan
 6. **Ruang status** (*state space*): himpunan semua simpul
 7. **Ruang solusi:** himpunan status solusi
- **Solusi:** lintasan dari akar ke simpul solusi

Teknik pencarian menggunakan algoritma traversal graf dapat dievaluasi dengan properti-properti berikut [2]:

1. **Completeness:** terkait apakah teknik pencarian menjamin ditemukannya solusi (jika ada)
2. **Optimality:** terkait apakah teknik pencarian menjamin mendapatkan solusi yang optimal
3. **Time Complexity:** waktu yang diperlukan untuk mencapai simpul solusi
4. **Space Complexity:** ruang memori yang diperlukan selama melakukan pencarian
5. Kompleksitas waktu dan ruang diukur dengan menggunakan istilah berikut,
 - b (*branching factor*): maksimum pencabangan yang mungkin dari suatu simpul
 - d (*depth*): kedalaman dari solusi terbaik (dengan cost terendah, sesuai definisi cost pada masalah)
 - m : maksimum kedalaman dari ruang status (mungkin memiliki nilai ∞)

2.1.1 Depth First Search

Depth First Search yaitu traversal graf secara mendalam yang dapat digambarkan dengan algoritma rekursif berikut ini (traversal dimulai dari simpul v) [1],

1. Kunjungi simpul v (simpul saat ini)
2. Kunjungi simpul w yang bertetangga dengan simpul v .
3. Ulangi DFS mulai dari simpul w tersebut

4. Ketika mencapai simpul u sedemikian sehingga semua simpul yang bertetangga dengannya telah dikunjungi, pencarian dirunut-balik (*backtrack*) ke simpul terakhir yang dikunjungi sebelumnya dan mempunyai simpul yang belum dikunjungi.
5. Pencarian berakhir bila tidak ada lagi simpul yang belum dikunjungi yang dapat dicapai dari simpul yang telah dikunjungi.

Struktur data yang dibutuhkan untuk DFS secara rekursif yaitu hanya,

1. Tabel boolean yang melacak simpul yang telah dikunjungi, dengan deskripsi:
 - $visited : array[l..n]$ of boolean
 - $visited[i] = true$ jika simpul i sudah dikunjungi
 - $visited[i] = false$ jika simpul i belum dikunjungi

DFS dapat digambarkan dengan *pseudo-code* berikut [1],

Algorithm 1 Mengunjungi seluruh simpul graf dengan DFS

Require: Tabel boolean $visited$ telah diinisialisasi dan dapat diakses

procedure DFS(v)

Input: v : simpul yang dikunjungi saat ini

 PROCESS(v) ▷ Proses simpul sesuai kebutuhan

for $u \in \text{NEIGHBOUR}(v)$ **do**

if not $visited[u]$ **then**

 DFS(u)

end if

end for

end procedure

Simpul-simpul yang dikunjungi pada traversal graf secara DFS dapat digambarkan sebagai pohon, dinamakan pohon DFS [1].

Depth First search dapat menemukan jalur dengan jarak terpendek khusus untuk graf berbentuk pohon (karena hanya ada satu kemungkinan jalur antar simpul) [3].

Depth First Search memiliki properti berikut [1] [3],

- Completeness? Ya (selama nilai *branching factor* terbatas)

- Optimality? Tidak
- Kompleksitas waktu: $O(b^m)$ atau $O(m + n)$ dengan b *branching factor*, m jumlah simpul, dan n jumlah sisi
- Kompleksitas ruang: $O(bm)$ atau $O(m)$ dengan b *branching factor*, m jumlah simpul
- Memiliki heuristik untuk kurang baik dalam kompleksitas waktu tetapi lebih baik dalam kompleksitas ruang
- Efisien tetapi tidak menjamin solusi dengan langkah minimum pada graf umum

DFS dapat memilih langkah yang salah, sehingga path panjang bahkan *infinite*. Pemilihan langkah sangat penting sehingga dapat dikembangkan teknik pencarian khusus seperti Depth Limited Search (DLS) dan Iterative Deepening Search (IDS) [2].

Depth Limited Search yaitu melakukan DFS dengan syarat tambahan, *backtrack* harus dilakukan jika telah mencapai suatu jarak n dari simpul awal. Sedangkan Iterative Deepening Search yaitu melakukan DLS dengan jarak n yang diatur menaik secara iteratif sampai solusi ditemukan [2].

DLS dapat digambarkan dengan *pseudo-code* berikut ini [2],

Algorithm 2 Mengunjungi seluruh simpul graf dengan DLS

Require: Tabel boolean *visited* telah diinisialisasi dan dapat diakses

function DLS(*v*, *limit*)

Input: *v*: simpul yang dikunjungi saat ini

limit: batas kedalaman maksimal dari simpul awal

Output: solusi dari permasalahan jika ditemukan

$result \leftarrow \text{PROCESS}(v)$

$\triangleright$ Proses simpul sesuai kebutuhan

if $result = \text{success}$ **then**

return $result$

end if

for $u \in \text{NEIGHBOUR}(v)$ **do**

if not $visited[u]$ **and** $\text{DEPTH}(u) \leq limit$ **then**

$result \leftarrow \text{DLS}(u)$

if $result = \text{success}$ **then**

return $result$

end if

end if

end for

end function

DLS memiliki properti sebagai berikut [2],

- Completeness? Tidak
- Optimality? Tidak
- Kompleksitas waktu: $O(b^n)$, dengan b *branching factor* dan n batas jarak
- Kompleksitas ruang: $O(bn)$, dengan b *branching factor* dan n batas jarak.

IDS dijalankan melalui fungsi *DLS* dari algoritma 2. IDS dapat digambarkan melalui *pseudo-code* berikut ini [2],

Algorithm 3 Mengunjungi seluruh simpul graf dengan IDS

Require: Tabel boolean *visited* telah diinisialisasi dan dapat diakses

function IDS(v , $limit$)

Input: v : simpul awal yang akan dikunjungi

$limit$: batas kedalaman maksimal dari simpul awal

Output: solusi dari permasalahan jika ditemukan

$i \leftarrow 0$

$result \leftarrow \text{DLS}(v, i)$

while $i + 1 \leq limit$ **and** $result \neq \text{success}$ **do**

$i \leftarrow i + 1$

$result \leftarrow \text{DLS}(v, i)$

end while

return $result$

end function

IDS memiliki propooerti sebagai berikut [2],

- Completeness? Ya, jika b terbatas
- Optimality? Ya, jika langkah sama dengan biaya
- Kompleksitas waktu: $O(b^d)$, dengan b *branching factor* dan d kedalaman dari solusi terbaik
- Kompleksitas ruang: $O(bd)$, dengan b *branching factor* dan d kedalaman dari solusi terbaik

2.1.2 Breadth First Search

Breadth First Search adalah teknik pencarian graf secara mendalam yang dapat digambarkan dengan algoritma berikut ini (traversal dimulai dari simpul v),

1. Kunjungi simpul v
2. Kunjungi semua simpul yang bertetangga dengan simpul v terlebih dahulu.
3. Kunjungi simpul berikutnya yang belum dikunjungi namun bertetangga dengan simpul-simpul yang telah dikunjungi, demikian seterusnya.

Breadth First Search dapat menemukan jalur terpendek pada graf umum dengan syarat graf tersebut merupakan graf tidak berbobot [4].

Simpul-simpul yang dikunjungi pada traversal graf secara BFS dapat digambarkan sebagai pohon, dinamakan pohon BFS [1].

Untuk melakukan BFS, diperlukan beberapa struktur data [1]:

1. Matriks ketetanggaan $A = [a_{ij}]$ yang berukuran $n \times n$,
 - $a_{ij} = 1$, jika simpul i dan simpul j bertetangga,
 - $a_{ij} = 0$, jika simpul i dan simpul j tidak bertetangga.
2. Antrian q untuk menyimpan simpul yang telah dikunjungi.
3. Tabel boolean untuk melacak simpul yang telah dikunjungi,
 - *visited* : *array*[$l..n$] of boolean
 - *visited*[i] = *true* jika simpul i sudah dikunjungi
 - *visited*[i] = *false* jika simpul i belum dikunjungi.

BFS dapat digambarkan dengan *pseudo-code* berikut [1],

Algorithm 4 Mengunjungi seluruh simpul graf dengan BFS

Require: Tabel boolean *visited* telah diinisialisasi dan dapat diakses

Require: Struktur data antrian simpul Q yang terinisialisasi dan kosong

procedure BFS(v)

Input: v : simpul awal yang dikunjungi

while not EMPTY(Q) **do**

$x \leftarrow$ POP(Q)

for $y \in$ NEIGHBOUR(x) **do**

if not *visited*[y] **then**

 PROCESS(y)

 ▷ Proses simpul sesuai kebutuhan

 PUSH(Q , y)

end if

end for

end while

end procedure

Selain itu, BFS juga memiliki properti-properti berikut ini [2] [4],

- Completeness? Ya (selama nilai b terbatas)

- Optimality? Ya, jika langkah sama dengan ongkos
- Kompleksitas waktu: $O(b^d)$ atau $O(m+n)$ dengan b *branching factor*, d kedalaman dari solusi terbaik, m jumlah simpul dan n jumlah sisi.
- Kompleksitas ruang: $O(b^d)$ atau $O(m)$ dengan b *branching factor*, d kedalaman dari solusi terbaik, dan m jumlah simpul
- Memiliki heuristik untuk kurang baik dalam kompleksitas ruang.

2.2 Lowest Common Ancestor

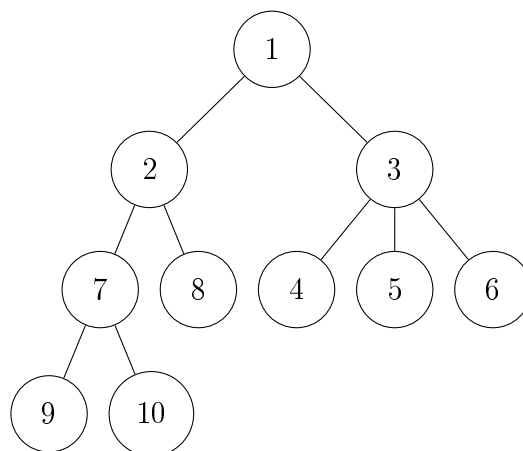
Misalnya $G = (V, E)$ adalah sebuah graf berbentuk tree dengan V adalah himpunan simpul dan E adalah himpunan sisi, akar dari G adalah suatu simpul tertentu yang dipilih sebagai simpul paling atas dengan *depth* bernilai nol, semua simpul lain pada G akan memiliki *depth* yang didefinisikan sebagai jarak DFS dari simpul tersebut ke simpul akar [5].

Dapat didefinisikan sebuah *ancestor* dari simpul $v \in V$ sebagai semua simpul pada jalur dari akar G ke simpul v (termasuk simpul akar dan simpul v itu tersendiri) [5].

Misal terdapat simpul $u, v \in V$ dan S adalah himpunan seluruh simpul yang merupakan *ancestor* dari u dan *ancestor* dari v , Lowest Common Ancestor (LCA) dari pasangan simpul (u, v) adalah sebuah simpul $w \in S$ dengan *depth* terbesar (dengan kata lain, memiliki posisi "terbawah" atau paling jauh dari akar) [5].

Sebagai contoh,

G :



- $LCA(8, 9) = 2$

- $LCA(4, 6) = 3$
- $LCA(2, 7) = 2$
- $LCA(5, 5) = 5$
- $LCA(6, 10) = 1$

Permasalahan pencarian LCA dapat direduksi menjadi permasalahan Range Minimum Query (RMQ) yang juga merupakan salah satu permasalahan klasik dalam Strategi Algoritma [6]. Selain itu ada banyak teknik untuk mencari solusi LCA, diantaranya yaitu [5]:

- Bruteforce Naif
- DFS dengan dua pointer
- Binary Lifting
- Algoritma Farach-Colton and Bender
- Algoritma Offline Tarjan

2.3 Binary Lifting

Misalnya $G = (V, E)$ adalah sebuah graf berbentuk tree dengan V adalah himpunan simpul, E adalah himpunan sisi dan N adalah banyaknya simpul pada G .

Binary Lifting adalah teknik efisien untuk menemukan *ancestor* ke- k ($k \geq 0$) dari setiap simpul $v \in V$ menggunakan paradigma *dynamic programming*. Teknik ini dilakukan dengan melakukan prekomputasi setiap *ancestor* ke- i ($i \in \{2^0, 2^1, \dots, 2^{\lceil \log N \rceil}\}$) dari setiap simpul $v \in V$ yang dapat dilakukan menggunakan DFS dalam kompleksitas waktu $O(N \log N)$, kemudian setiap query untuk mencari *ancestor* ke- k dari sebuah simpul v akan bisa dijawab dalam kompleksitas waktu $O(\log k)$ [8].

Implementasi Binary Lifting dapat dilakukan menggunakan struktur data [8],

$ancestor[1..N][0..\lceil \log N \rceil]$: array of array of nodes

Misalkan terdapat suatu simpul v yang memiliki *parent* p . Relasi rekurens yang dapat digunakan untuk melakukan prekomputasi pada v yaitu [7],

$$ancestor[v][i] = \begin{cases} p & i = 0 \\ ancestor[ancestor[v][i-1]][i-1] & i > 0 \end{cases}$$

Prekomputasi harus dilakukan pada setiap simpul v di G dan harus dipastikan bahwa sebelum melakukan prekomputasi pada suatu simpul, semua *ancestor* simpul tersebut harus sudah di-prekomputasi (jika tidak, maka nilai prekomputasi untuk simpul tersebut tidak benar), sehingga cocok untuk menggunakan DFS untuk traversal yang digunakan. Program prekomputasi dan pencarian *ancestor* ke- k dapat digambarkan melalui *pseudo-code* berikut ini [7] [8],

Algorithm 5 Prekomputasi pada Binary Lifting

procedure DFS_PRECOMPUTE($v, p, ancestor, N$)

Input: v : simpul yang dikunjungi saat ini
 p : *parent* dari v
 $ancestor$: tabel prekomputasi setiap simpul
 N : jumlah simpul pada tree

$ancestor[v][0] = p$

for $i \leftarrow 1$ to $\lceil \log N \rceil$ **do**

$ancestor[v][i] = ancestor[ancestor[v][i-1]][i-1]$

end for

for $u \in \text{NEIGHBOUR}(v)$ **do**

if $u \neq p$ **then**

DFS_PRECOMPUTE($u, v, ancestor$)

end if

end for

end procedure

procedure PRECOMPUTE($tree, ancestor$)

Input: $tree$: tree yang akan di-prekomputasi
 $ancestor$: tabel prekomputasi setiap simpul

$root \leftarrow \text{GETROOT}(tree)$

$N \leftarrow \text{GETNODESCOUNT}(tree)$

DFS_PRECOMPUTE($root, root, ancestor, N$)

▷ Akar tree dianggap

parent-nya sendiri

end procedure

Algorithm 6 Teknik Binary Lifting untuk menemukan *ancestor* ke- k suatu simpul

Require: tabel prekomputasi simpul telah diisi dan berupa variabel global *ancestor*

Require: jumlah simpul dalam tree berupa variabel global N

function KTH_ANCESTOR(v, k)

Input: v : simpul yang akan dicari simpul *ancestor*-nya

k : jarak ke simpul *ancestor* yang dicari

Output: simpul yang merupakan *ancestor* ke- k dari v

$u \leftarrow v$

for $i \leftarrow 0$ to $\lceil \log N \rceil$ **do**

if bit ke- i dihitung dari Least Significant Bit pada k bernilai 1 **then**

$u \leftarrow \text{ancestor}[u][i]$

end if

end for

return u

end function

Perhatikan bahwa Binary Lifting memiliki kompleksitas memori sebesar $O(N \log N)$.

2.3.1 Pemanfaatan Binary Lifting pada LCA

Binary Lifting dapat digunakan sebagai salah satu teknik dalam mencari solusi LCA yang efisien. Setelah prekomputasi Binary Lifting dilakukan, sebuah query LCA dapat dihitung dalam kompleksitas waktu $O(\log N)$ [7].

Misal terdapat kueri untuk menemukan $LCA(x, y)$, dengan memakai fungsi $kth_ancestor(x, k)$ yang telah dijelaskan pada Algoritma 6, kita dapat dilakukan langkah-langkah algoritmik berikut untuk menemukan LCA [8]:

1. Asumsikan $depth(x) \geq depth(y)$, jika tidak maka kembalikan hasil $LCA(y, x)$ (hasil akan identik)
2. Ambil $k = depth(x) - depth(y)$ dan $x = kth_ancestor(x, k)$, dengan kata lain kita akan "menaikkan" x sehingga x dan y berada pada $depth$ yang sama
3. Jika $x = y$ maka kembalikan simpul x sebagai hasil kueri, jika tidak maka pergi ke langkah selanjutnya
4. Ambil $S = \{2^j \mid 2^j \leq depth(x), j \in \{0, 1, 2, \dots\}\}$, Mulai dari nilai $i \in S$ yang terbesar hingga terkecil lakukan: Jika $kth_ancestor(x, i) \neq kth_ancestor(y, i)$

maka ambil $x = kth_ancestor(x, i)$ dan $y = kth_ancestor(y, i)$ (Dengan kata lain, kita "menaikkan" x dan y bersamaan selama hasil "penaikkan" itu tidak menghasilkan $x = y$)

5. Sampai tahap ini, x dan y berada pada posisi "paling atas" sehingga $x \neq y$, karena suatu simpul hanya memiliki satu *parent* maka akibatnya $parent(x) = parent(y) = LCA(x, y)$. Kembalikan simpul $kth_ancestor(x, 0)$ sebagai hasil kueri.

Untuk lebih jelasnya, LCA menggunakan Binary Lifting dapat dipaparkan melalui *pseudo-code* berikut [8],

Algorithm 7 Solusi kueri LCA menggunakan teknik Binary Lifting

Require: tabel prekomputasi simpul telah diisi dan berupa variabel global *ancestor*

Require: jumlah simpul dalam tree berupa variabel global N

function LCA(x, y)

Input: x : simpul pertama yang akan dicari LCA-nya

y : simpul kedua yang akan dicari LCA-nya

Output: simpul yang merupakan $LCA(x, y)$

if DEPTH(x) < DEPTH(y) **then**

return LCA(y, x)

end if

$k \leftarrow \text{DEPTH}(x) - \text{DEPTH}(y)$

$z \leftarrow \text{KTH_ANCESTOR}(x, k)$

if $z = y$ **then**

return z

end if

for $i \leftarrow \lceil \log N \rceil$ **to** 0 **do**

if $i \leq \text{DEPTH}(z)$ **and** $\text{KTH_ANCESTOR}(y, i) \neq \text{KTH_ANCESTOR}(z, i)$ **then**

$y \leftarrow \text{KTH_ANCESTOR}(y, i)$

$z \leftarrow \text{KTH_ANCESTOR}(z, i)$

end if

end for

return $\text{KTH_ANCESTOR}(z, 0)$

end function

2.4 HTML dan DOM Tree

HTML (*HyperText Markup Language*) adalah bahasa formal bertipe *markup language* yang digunakan untuk mendefinisikan arti dan struktur sebuah konten web. HTML biasanya digunakan bersamaan dengan teknologi web lain seperti CSS, JavaScript, WebAssembly, dan lain-lain. HTML menggunakan elemen spesial untuk mendeskripsikan konten, elemen tersebut dikatakan sebagai HTML Element [9].

Sebuah HTML Element dibedakan dari teks biasa dengan menggunakan *tags*, yaitu nama elemen tersebut yang dikelilingi oleh karakter '`<`' dan '`>`'. Nama dari HTML Element bersifat *case-insensitive*. Sebuah dokumen HTML selalu memiliki elemen pertama dengan nama elemen "html". Adapun beberapa elemen spesial lainnya yang terdefinisi dalam spesifikasi HTML yaitu [9],

```
<head>
<title>
<body>
<p>
<div>
<footer>
```

Terdapat juga bahwa sebuah *tag* dari elemen HTML harus ditutup dengan *tag* dengan nama elemen yang sama, tetapi memiliki simbol '/' sebelum simbol '>' penutup. Sehingga kebanyakan elemen HTML dideskripsikan melalui dua buah *tag*, yang terdiri dari *tag* pembuka dan penutup. Tetapi terdapat juga elemen HTML yang bersifat *self-closing*, yaitu hanya terdiri dari sebuah *tag* yang berlaku sebagai pembuka dan penutup secara bersamaan [9].

Selain itu, sebuah elemen HTML tidak hanya memiliki nama saja, tetapi dapat juga memiliki atribut-atribut tambahan. Atribut-atribut tersebut dapat dideskripsikan sebagai kumpulan pasangan nama atribut beserta nilai atribut yang dipisahkan oleh simbol '=', kemudian antara atribut yang berbeda dipisahkan oleh karakter *whitespaces*. Atribut juga bisa saja tidak memiliki nilai sehingga hanya memiliki nama atribut saja [9].

Semua atribut suatu elemen harus tertampung pada *tag* pembuka (atau pembuka-penutup pada *self-closing tag*), atau dalam kata lain harus muncul setelah nama *tag* dan sebelum karakter penutup '>'. Jika terdapat lebih dari satu nama atribut yang sama, maka nilai atribut yang pertama muncul-lah yang akan diasosiasikan dengan nama atribut tersebut. Adapun contoh-contoh *tag* yang mendeskripsikan elemen HTML diantaranya [9]:

```

<head> </head>
<p> Hello World! </p>
<div level="1"> <div level="2"> </div> </div>
<input type="radio" id="input-1" name="choices" checked />

```

Dokumen HTML yang mendeskripsikan konten web dapat dikonversi menjadi sebuah DOM (*Document Object Model*), DOM tersebut merepresentasikan konten pada dokumen dalam bentuk struktur data tree, dimana setiap simpul adalah elemen HTML. Metode ini memungkinkan konten web untuk diakses dan dimodifikasi secara programmatik, modular dan memiliki skalabilitas tinggi [10].

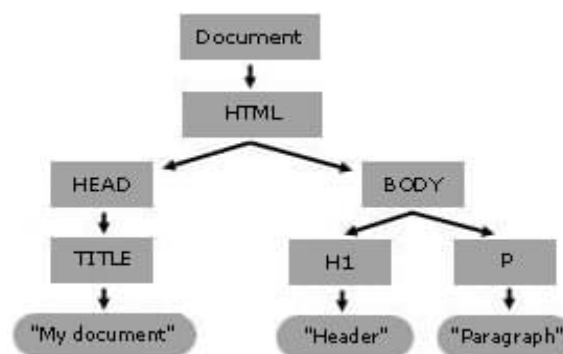
Sebagai contoh, perhatikan dokumen HTML berikut ini:

```

<html lang="en">
  <head>
    <title>My Document</title>
  </head>
  <body>
    <h1>Header</h1>
    <p>Paragraph</p>
  </body>
</html>

```

Dokumen tersebut dapat digambarkan sebagai DOM Tree berbentuk berikut ini [10],



Gambar 4: Ilustrasi contoh DOM Tree HTML

Adapun untuk mengubah dokumen HTML menjadi sebuah DOM Tree diperlukan tahapan-tahapan berikut [11],

1. Melakukan dekodifikasi byte yang datang dari sumber pengirim dokumen.

2. Melakukan *pre-processing* pada masukan yang datang, dengan cara menentukan *encoding*, menghilangkan karakter tertentu, ataupun menentukan validitas berdasarkan karakter kontrol tertentu.
3. Melakukan tokenisasi menjadi 69 token yang terdefinisi
4. Membentuk DOM Tree sesuai token yang ada dengan menggunakan struktur data *stack*
5. Melakukan eksekusi *script* pada dokumen jika ada, dan merubah bentuk DOM Tree sesuai *script* yang terdefinisi

2.5 CSS Selector

CSS (*Cascading Style Sheet*) merupakan bahasa formal yang berbasis aturan (*rule-based*). CSS dapat digunakan untuk memberikan *style* pada sebuah dokumen HTML, dimana *style* tersebut dapat diinterpretasikan oleh sebuah *web browser*. Bahasa CSS diorganisasikan menjadi beberapa modul fungsionalitas, misalnya modul *styling* teks, latar belakang, animasi, tabel, dan lain-lain [12].

Dalam mendeskripsikan aturan pemberian *style* pada sebuah dokumen HTML, CSS memanfaatkan struktur HTML yang dapat dibuat menjadi sebuah *tree*, lebih tepatnya DOM Tree. Umumnya CSS digunakan dengan memberikan spesifikasi terhadap aturan *styling* tertentu untuk himpunan simpul DOM Tree tertentu. Untuk menyatakan himpunan simpul tersebut digunakan CSS Selector, kemudian himpunan tersebut akan diberikan beberapa CSS Rule yang berbentuk deklarasi himpunan *property* dan *value*. [12]

CSS Selector adalah salah satu modul fungsionalitas CSS yang memiliki fungsi *pattern-matching* untuk memberikan spesifikasi himpunan simpul DOM Tree yang dipilih. Ada banyak sekali komponen dari modul CSS Selector, tipe-tipe komponen selector umumnya dapat kita kategorikan menjadi [13],

1. Basic Selector

- **Type Selector**, memilih simpul berdasarkan nama dari *tag* HTML.
- **Universal Selector** (*), memilih simpul apa saja dari DOM Tree.
- **Class Selector** (.), memilih simpul berdasarkan kelas elemen tertentu.
- **ID Selector** (), memilih simpul berdasarkan ID unik elemen tertentu.

2. **Attribute Selector ([|])**, selektor yang mencocokkan nilai atribut dari sebuah *tag* HTML dengan berbagai variasi tertentu, misalnya *substring matching*, *prefix matching*, *suffix matching*, dan lain-lain.
3. **Pseudo-Class Selector (:)**, selektor yang mencocokkan simpul DOM Tree menggunakan informasi yang tidak terdapat langsung pada tree tersebut, misalnya mencocokkan elemen berdasarkan interaksi *mouse* atau *keyboard* pengguna.
4. **Pseudo-Element Selector (::)**, selektor yang mencocokkan simpul sehingga aturan *styling* dapat diberikan pada bagian tertentu elemen secara parsial, misalnya mencocokkan elemen sehingga baris pertama paragraf tertentu dapat memiliki *style*.
5. **Compound Selector**, gabungan beberapa selektor dasar menjadi rantai selektor yang lebih panjang, selektor dasar tersebut dapat memiliki Attribute Selector, Pseudo-Class Selector, ataupun Pseudo-Element Selector.
6. **Combinator**, merupakan operator penggabungan beberapa Compound Selector.
 - **Descendant ()**, menyatakan operator yang mencocokkan elemen di kanan sebagai keturunan dari elemen di kiri operator.
 - **Child (>)**, menyatakan operator yang mencocokkan elemen di kanan sebagai anak langsung dari elemen di kiri operator.
 - **Subsequent-Sibling ()**, menyatakan operator yang mencocokkan elemen di kanan sebagai elemen yang memiliki *parent* yang sama dengan elemen di kiri dan terletak setelahnya dalam urutan *pre-order traversal*.
 - **Next-Sibling (+)**, menyatakan operator yang mencocokkan elemen di kanan sebagai elemen pertama yang dikunjungi melalui *parent* kedua elemen tersebut dalam urutan *pre-order traversal*.
 - **Column (|)**, menyatakan operator yang mencocokkan simpul yang berada pada sebuah kolom tabel atau grid.
7. **Complex Selector**, merupakan gabungan beberapa Compound Selector yang digabungkan menggunakan Combinator sehingga menjadi rantai yang lebih kompleks.
8. **Selector List**, merupakan gabungan beberapa Complex Selector yang digabungkan menggunakan koma (',').

9. **Namespace Separator** (`'|'`), merupakan operator khusus yang membatasi pencocokan selektor lainnya berdasarkan *namespace* HTML tertentu, misalnya *namespace* SVG dan MathML.

Dokumen HTML yang dipadukan dengan CSS akan diinterpretasikan dengan langkah-langkah sebagai berikut [12],

1. Browser memuat dokumen HTML dan membangun sebuah DOM Tree.
2. Jika terdapat aturan CSS, maka browser melakukan pencocokkan CSS Selector, dengan tahapan tokenisasi, pembentukan struktur selektor, dan pencocokan dengan *tree traversal*.
3. Semua aturan CSS yang ditemukan akan dicocokkan berdasarkan CSS Selector yang sesuai.
4. Aturan CSS akan diaplikasikan ke DOM Tree sehingga terbentuk Render Tree, yang akan digunakan Rendering Engine untuk membuat tampilan GUI.

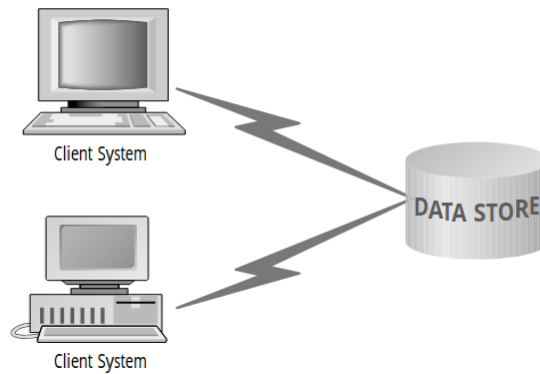
2.6 Client-Server Architecture

Arsitektur Client-Server adalah istilah umum yang biasa digunakan dalam arsitektur sistem modern untuk mendeskripsikan arsitektur aplikasi yang membagi pemrosesan ke dalam lebih dari satu proses dan umumnya juga dalam perangkat keras yang berbeda [17].

Pembagian tersebut dilakukan berdasarkan spesialisasi dan kebutuhan, misalnya Web Browser (client) akan menampilkan antarmuka sedangkan Webserver (Server) akan melakukan manajemen *networking* dan menjamin penyaluran data. [17]

Salah satu contoh arsitektur Client-Server yang paling sederhana yaitu *two-tier architecture*. Arsitektur jenis ini umumnya memberikan antarmuka sederhana bagi pengguna untuk mendapatkan informasi yang tersentralisasi tanpa memberikan interpretasi data atau pemrosesan data yang intensif. Contoh aplikasi yang memakai arsitektur ini yaitu [17],

- **Email**
- **Telnet**
- **FTP** (File Transfer Protocol)
- **Gopher**



Gambar 5: Ilustrasi sistem *two-tier architecture*

- **Web.**

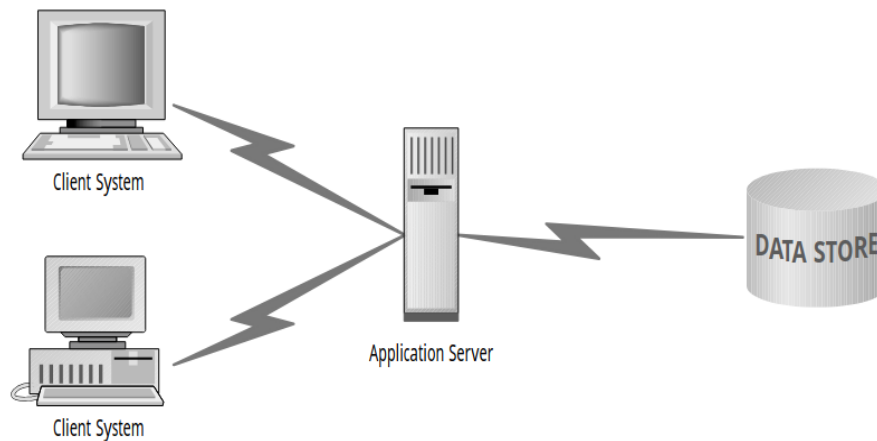
Arsitektur dengan tipe *two-tier architecture* juga memiliki limitasi yang perlu dipikirkan oleh arsitek sistem [17],

1. **Fat Clients**, yaitu permasalahan dimana dengan semakin berkembangnya teknologi maka fungsionalitas client akan berkembang sangat kompleks sehingga Server tidak bisa mengikuti perkembangan dengan baik
2. **Object Reuse**, yaitu penggunaan data tersentralisasi yang "diulangi" sehingga terdapat duplikat dan akibatnya akan sulit menjaga integritas data, hal ini sering terjadi karena fungsionalitas antar komponen pada sistem *two-tier* yang berbeda cenderung mirip.

Untuk meningkatkan desain sistem, dapat digunakan *three-tier architecture* yang mengisolasi pemrosesan data pada suatu lapisan baru yang berbeda dengan lapisan inti program. Lapisan tersebut biasanya disebut sebagai lapisan basis data. [17]

2.7 Parallel Computing

Paradigma komputasi parallel (*parallel computing*) adalah teknik komputasi dengan cara menggunakan lebih dari satu sumber daya komputasi di saat yang bersamaan. Paradigma ini dapat digunakan untuk menyelesaikan masalah dengan cara memecah permasalahan menjadi upa-permasalahan lebih kecil yang independen satu sama lain dan hasilnya dapat digabungkan menjadi solusi global, kemudian upa-permasalahan dibagi kepada sumber daya komputasi berbeda, dan penemuan solusi dilakukan bersamaan dengan teknik sinkronisasi tertentu [15].



Gambar 6: Ilustrasi sistem *three-tier architecture*

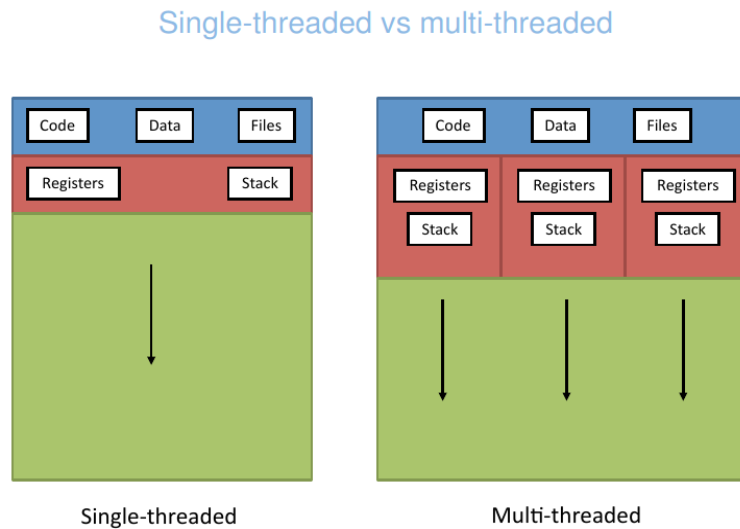
Kemampuan komputasi parallel dapat dihasilkan dengan beberapa cara misalnya [15],

- Lebih dari satu unit fungsional (cache L1, cache L2, cabang, prefetch, decode, floating-point, pemrosesan grafis (GPU), integer, dll.)
- Lebih dari satu unit/*core* eksekusi
- Lebih dari satu *core* perangkat keras
- Jaringan komputer yang terdiri dari banyak komputer dan berdiri sendiri untuk membuat cluster komputer parallel yang lebih besar.

Aplikasi komputasi parallel dalam suatu perangkat keras merupakan tanggung jawab sistem operasi perangkat keras tersebut. Umumnya sistem operasi akan mengimplementasikan mekanisme *context switching* yang dapat memberikan manfaat *concurrency* dan *parallelism*. *Concurrency* yaitu kemampuan untuk mengerjakan banyak proses dalam waktu yang sama dengan cara berganti proses secara cepat, tanpa sumber daya komputasi tambahan. Sedangkan *Parallelism* adalah kemampuan untuk mengerjakan banyak proses dalam waktu yang sama dengan cara menggunakan sumber daya komputasi lainnya secara maksimal. [15] [16]

Aplikasi *parallel computing* yaitu *parallel programming*. Dalam pembuatan program komputer modern, unit terkecil eksekusi yang biasa digunakan yaitu *thread*, kumpulan *thread* yang memiliki ruang memori yang sama akan membentuk *process*. Penggunaan *multi-threading* dibandingkan *multi-processing* lebih diutamakan karena, *context switch* antar *thread* lebih murah secara komputasional. Selain itu *thread* juga saling berbagi bagian data *heap* dan *global* sehingga dapat lebih mudah

untuk sinkronisasi. [16]



Gambar 7: Ilustrasi *multi-threading* pada komputer

Sinkronisasi perlu dilakukan antar unit eksekusi yang saling berbagi data pada alamat memori yang sama, jika tidak maka akan terjadi *race condition*. Teknik sinkronisasi yang dapat dilakukan antar unit eksekusi yaitu diantaranya [15],

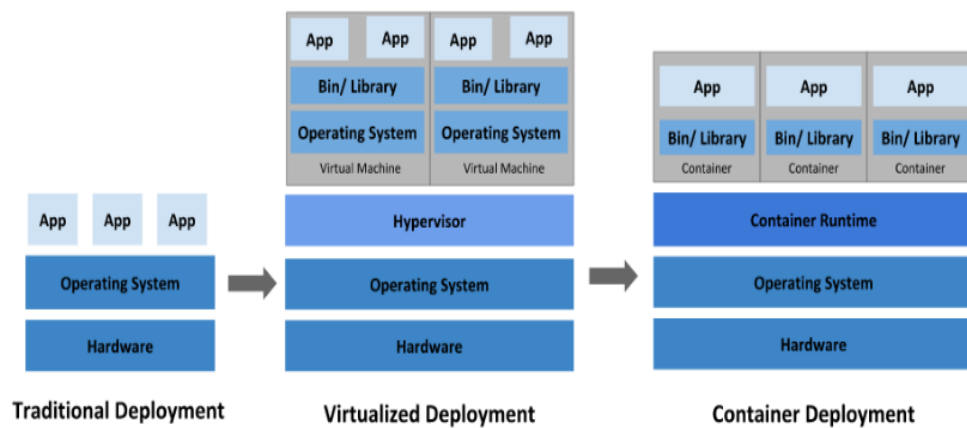
- **Barrier**, sebuah unit komputasi akan melakukan sebuah pekerjaan hingga mencapai suatu "batasan" dan akan berhenti
- **Lock / semaphore**, digunakan untuk melindungi data global dengan mekanisme kunci sehingga hanya satu unit komputasi yang bisa memodifikasi data pada suatu waktu
- **Synchronous communication operations**, unit komputasi saling berkomunikasi satu sama lain untuk menyelesaikan pekerjaan tertentu

2.8 Virtualisasi dan Kontainerisasi

Virtualisasi adalah teknik mengabstraksi dan mengatur sumber daya dari perangkat keras sehingga perangkat lunak mendapat sumber daya virtual dibanding sumber daya asli. Fitur sistem operasi yang dapat mem-virtualisasikan sumber daya seperti fitur *time-sharing* dan *virtual memory*, menginisiasikan munculnya teknologi virtualisasi dengan *hypervisor*. Hasilnya, muncullah teknologi *virtual machine* yang memungkinkan manajemen lebih dari satu sistem operasi pada suatu mesin [19].

Teknologi virtualisasi memberikan keuntungan yang besar karena berhasil meningkatkan nilai perangkat keras. Sehingga pada awalnya menjadi *state-of-the-art* dalam urusan *deployment and operation*. Tetapi teknologi virtualisasi dengan *hypervisor* memakan banyak memori dan memiliki *overhead* dari manajemen banyak sistem operasi, sehingga dikembangkan teknologi kontainerisasi [18].

Kontainerisasi adalah cara virtualisasi yang lebih ringan dan cepat, dengan cara menggabungkan seluruh kode dan data yang berkaitan ke dalam suatu proses tertentu yang akan mengatur isolasi menggunakan fitur dari sistem operasi *host*. Pada dasarnya, sistem operasi *host* tersebut-lah yang harus memberikan antarmuka untuk mengisolasi sumber daya [19].

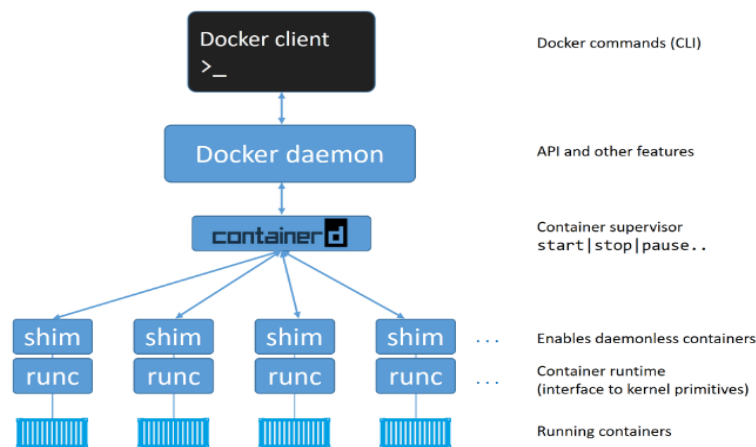


Gambar 8: Ilustrasi virtualisasi dan kontainerisasi

Spesifikasi kontainerisasi diatur oleh OCI (Open Container Initiative). Salah satu teknologi kontainerisasi yang banyak digunakan yaitu Docker Engine yang memiliki tujuan untuk memudahkan pembuatan kontainer menggunakan *image*. Arsitektur Docker Engine sangat modular dan terdiri dari lapisan-lapisan [18]:

1. **Docker client**, antarmuka bagi pengguna untuk menjalankan fitur Docker Engine (biasanya melalui CLI)
2. **Docker daemon**, menerima *request* dari Docker Client dan meneruskannya melalui API ke komponen lain
3. **containerd**, proses modular yang mengatur daur hidup dari sebuah kontainer
4. **shim**, proses yang menjaga agar integritas kontainer tetap terjaga walaupun terjadi masalah dengan Docker ataupun containerd

5. **runc**, proses yang memiliki tanggung jawab untuk membuat kontainer berdasarkan spesifikasi OCI dan mengakses primitif isolasi dari fitur sistem operasi



Gambar 9: Ilustrasi arsitektur Docker Engine

Docker Engine menggunakan sebuah format *image* untuk mendeskripsikan sebuah kontainer. Template tersebut akan diinterpretasikan oleh Docker Engine menjadi beberapa lapisan read-only yang modular. Lapisan-lapisan tersebut didesain sesuai spesifikasi OCI dan akan menjadi bahan bagi **runc** untuk membuat *runtime* kontainer [18].

Untuk membuat sebuah Docker Image, beberapa contoh perintah primitif yang dapat digunakan yaitu [18]:

- **FROM**: untuk menggunakan basis lapisan untuk image saat ini
- **RUN**: untuk menjalankan eksekusi proses tertentu saat proses pembuatan kontainer
- **COPY**: untuk menyalin file tertentu dari *host* ke kontainer saat proses pembuatan kontainer
- **WORKDIR**: untuk menentukan *working directory* proses utama kontainer
- **EXPOSE**: untuk menentukan nomor *port* yang di-ekspos dari *network namespace* kontainer ke *host*
- **CMD**: untuk menentukan *command* eksekusi program yang akan menjadi proses utama dari kontainer dan lain sebagainya.

3 Aplikasi DFS dan BFS

Pada permasalahan ini kita diberikan sebuah DOM Tree dan CSS Selector yang mendeskripsikan subhimpunan dari himpunan semua simpul DOM Tree tersebut. Pada dasarnya, kita perlu mencari subhimpunan tersebut melalui algoritma traversal graf klasik, dimana traversal dilakukan dengan mengecek sebuah fungsi validasi pada setiap simpul.

Algoritma DFS dan BFS dapat sangat berguna disini, karena algoritma tersebut memenuhi properti *completeness* yang menjamin solusi subhimpunan pasti ditemukan.

3.1 Perancangan DFS dan BFS

Perancangan algoritma DFS dan BFS perlu diperhatikan dengan tepat karena perlu memperhatikan hal-hal berikut,

- Perilaku Multi-threading
- Perilaku Combinator
- Perilaku Selector List dibanding Complex Selector biasa

3.1.1 Perilaku Multi-Threading

Terdapat tantangan tersendiri dalam implementasi traversal DOM Tree. Umumnya desain implementasi yang paling mudah (dan naif) yaitu dengan membuat implementasi DFS secara rekursif, dimana pada setiap rekursi akan dibuat thread yang baru.

Akan tetapi cara tersebut sangat buruk karena untuk memanfaatkan paralelisme secara maksimal, jumlah thread yang harus dibuat harus sama dengan jumlah *core* pada CPU pengguna.

Sehingga pilihan desain paling tepat yaitu untuk membuat algoritma iteratif menggunakan struktur data global yang *thread-safe*, struktur data global tersebut akan menyimpan sebuah struktur yang mendeskripsikan tugas yang bisa dilakukan sebuah *thread* saat mengunjungi DOM Tree tertentu.

Sehingga konsep algoritma yang akan digunakan yaitu,

- Sebuah thread utama akan menginisialisasi semua struktur data, dan memasukkan "tugas" pertama pada struktur data global
- Setiap thread anak akan berusaha mengambil "tugas" dari struktur data global (tugas untuk mengunjungi suatu simpul dan memprosesnya)
- Tugas yang dilakukan thread anak akan juga memasukkan "tugas" baru ke struktur data global (yaitu tugas untuk mengunjungi anak dari simpul saat ini)
- Setiap thread akan melakukan *busy-waiting* jika struktur data global kosong hingga dia mendapat tugas
- Setiap thread yang sudah mendapat tugas akan ditandai sebagai aktif, dan jika tugasnya selesai maka ditandai non-aktif
- Setiap thread akan berhenti jika struktur data global kosong dan tidak ada thread lagi yang ditandai sebagai aktif

Perlu diperhatikan bahwa tipe struktur data global akan mempengaruhi tipe traversal. Jika digunakan *stack* maka akan menjadi DFS, dan jika digunakan *queue* maka menjadi BFS.

3.1.2 Perilaku Combinator

CSS Selector dapat memiliki beberapa Combinator (mungkin nol dan mungkin lebih dari satu), perbedaan tipe Combinator akan menyebabkan perbedaan perilaku *branching* dalam traversal, baik itu untuk DFS ataupun BFS.

Setiap mengunjungi sebuah simpul dalam DFS/BFS, kita tentunya butuh selector saat ini yang akan di-cocokkan dengan simpul yang dikunjungi. Perlu diperhatikan beberapa observasi perilaku berikut ini terhadap combinator yang berada sebelum selector saat ini,

1. Untuk combinator tipe Descendant (' '), maka selector saat ini akan diteruskan ke semua simpul anak kecuali combinator dari selector selanjutnya juga Descendant (' ').
2. Untuk combinator tipe Child ('>'), jika simpul saat ini cocok dengan selector saat ini maka selector selanjutnya akan diteruskan ke simpul anak, selain itu tidak ada selector yang dilanjutkan ke simpul anak

3. Untuk combinator tipe SubsequentSibling (' '), maka selector saat ini akan diteruskan ke simpul saudara selanjutnya kecuali combinator dari selector selanjutnya juga SubsequentSibling (' ').
4. Untuk combinator tipe NextSibling ('+'), jika simpul saat ini cocok dengan selector saat ini maka selector selanjutnya akan diteruskan ke simpul saudara selanjutnya, selain itu tidak ada selector yang dilanjutkan ke simpul saudara.
5. Jika sebelumnya tidak ada combinator, maka lakukan perilaku seakan combinator sebelumnya adalah Descendant (' ').

Untuk mempermudah implementasi perilaku combinator yang berhubungan dengan *sibling* dari simpul saat ini, maka untuk setiap kunjungan ke suatu simpul akan disimpan *pointer* ke *parent* dari simpul tersebut, dan juga posisi urutan relatif simpul tersebut terhadap anak-anak lain dari *parent*.

3.1.3 Perilaku Selector List

Kueri untuk CSS Selector dapat berupa sebuah Selector List yang merupakan kumpulan CSS Selector yang dipisahkan oleh koma.

Perlu diperhatikan bahwa perilaku setiap simpul terhadap Selector List tersebut yaitu, jika setidaknya terdapat satu CSS Selector diantara Selector List tersebut yang cocok terhadap simpul tersebut, maka simpul dianggap cocok.

```
div > li, p ~ a + span, body
```

Oleh karena itu, operasi tersebut identik dengan operasi gabungan himpunan. Dimana hasil pencocokan untuk sebuah Selector List adalah gabungan dari operasi pencocokan setiap CSS Selector pada Selector List tersebut.

Observasi ini sangat penting karena terlihat bahwa operasi pencocokan dapat diparalelisasi untuk setiap CSS Selector yang dipisahkan koma, alias kita tidak perlu mencocokkan dari kiri ke kanan secara berurutan. Tetapi pada saat yang sama bisa langsung saja melakukan traversal BFS/DFS secara parallel terhadap setiap CSS Selector.

Perilaku ini dapat kita manfaatkan dengan memasukkan tugas ke struktur data global berupa perintah mengunjungi simpul akar sebanyak n kali, dimana n

adalah jumlah CSS Selector dalam Selector List, dan setiap CSS Selector dimasukkan ke salah satu dari tugas tersebut.

Sehingga algoritma traversal yang digunakan dapat digambarkan sebagai berikut,

1. Buat sebuah struktur yang mendeskripsikan perintah kepada sebuah thread untuk mengunjungi sebuah simpul, struktur tersebut berisi [*pointer* ke simpul tersebut, *pointer* ke *parent* simpul tersebut, posisi relatif simpul dalam urutan anak *parent*, Unit CSS Selector saat ini].
2. Buat sebuah struktur global yang mampu menyimpan daftar struktur perintah dari (1), pastikan struktur global *thread-safe* dan dapat dijangkau setiap thread.
3. Thread utama akan menginisiasi traversal dengan memasukkan perintah untuk mengunjungi simpul akar, dengan unit CSS Selector pertama, untuk setiap CSS Selector pada Selector List.
4. Setiap thread akan berlomba-lomba "menghabiskan" struktur data global, untuk setiap saat thread akan mengambil satu perintah dari situ dan menjalankannya (yang akan memasukkan perintah selanjutnya ke struktur global)
5. Jika struktur data global kosong, dan tidak ada thread yang sedang mengerjakan perintah, maka terminasi setiap thread dan kembalikan hasil traversal yang cocok.

Dengan membuat struktur global sebagai LIFO (*Last-in-first-out*) maka kita akan menjalankan DFS, sedangkan jika FIFO (*First-in-first-out*) maka akan berjalan BFS. Desain ini membuat kode cukup *reusable* dan modular.

3.2 Perancangan LCA

LCA sesuai yang dijelaskan pada bagian 2.2 secara teknis merupakan traversal jenis DFS, ada dua observasi yang dapat dilakukan untuk melakukan perancangan LCA,

1. setiap simpul pada tree pasti hanya memiliki satu *parent*, sehingga untuk menuju *ancestor* tertentu hanya terdapat satu jalur yang mendalam.
2. untuk menghitung $LCA(a, b, \dots, z)$ perlu dihitung $LCA(LCA(LCA(a, b), c), \dots)$ sehingga untuk pekerjaan mencari *LCA* dengan masukan lebih dari satu tetap diperlukan iterasi dua-pasang secara rekursif.

Karena kedua fakta tersebut maka jelas bahwa operasi LCA tidak bisa menjadi parallel, dan lebih baik di desain agar tidak menggunakan *mult-threading*. Sehingga untuk efisiensi dapat kita gunakan teknik *binary lifting* sesuai bagian 2.3.

Tetapi masalah yang harus dihadapi yaitu bagaimana cara memetakan isi simpul DOM Tree menjadi simpul berisi angka saja, hal ini perlu dilakukan karena teknik *binary lifting* membutuhkan simpul berupa elemen yang dapat diidentifikasi secara unik dan terurut, seperti bilangan bulat (hal tersebut konsekuensi dari paradigma teknik *binary lifting* yang merupakan *dynamic programming*).

Solusi terbaik yang dapat dilakukan yaitu untuk memetakan *pointer* setiap simpul DOM Tree menjadi sebuah bilangan bulat melalui struktur data HashTable, Hal ini dapat bekerja karena pastilah *pointer* untuk setiap objek dinamis akan berbeda, sehingga dapat menjadi pemetaan.

Setelah memecahkan masalah-masalah tersebut, desain algoritma dapat langsung mengikuti secara persis seperti teori yang telah dijelaskan pada 2.2 2.3.

4 Implementasi dan Pengujian

4.1 Implementasi

Implementasi secara garis besar dibagi menjadi dua buah proyek, yaitu server backend yang mengatur logika parsing dan traversal, kemudian ada aplikasi frontend dalam bentuk konten web yang akan menjadi antarmuka pengguna

4.1.1 Backend Server

Implementasi server backend dilakukan menggunakan bahasa pemrograman **Rust** dan *package manager/build tools* **Cargo**.

Digunakan *framework* eksternal yaitu **html5gum** dan **css_lexer**, dimana keduanya hanya membantu melakukan tokenisasi elemen-elemen HTML dan CSS. Selain itu digunakan teknik memori manajemen menggunakan tipe *built-in* Arc (*asynchronous reference counting*) pointer dari bahasa pemrograman **Rust**.

Detail implementasi meliputi beberapa file berikut:

- **async_util.rs**, file ini berisi struktur yang akan membantu dalam melakukan *multi-threading*.
- **css_selector.rs**, file ini berisi seluruh fungsionalitas *parser* CSS Selector yang dibuat.
- **html.rs**, file ini berisi seluruh fungsionalitas *parser* HTML yang dibuat.
- **matching.rs**, file ini berisi fungsi pembantu untuk pencocokan hubungan antara simpul DOM Tree dengan CSS Selector.
- **traversal.rs**, file ini berisi logika utama *multi-threaded* DFS dan BFS.
- **lca.rs**, file ini berisi logika utama teknik *binary lifting* dan LCA yang digunakan.
- **main.rs**, file ini berisi logika server utama yang akan menerima dan merespon request untuk mengirim aplikasi frontend dalam bentuk konten web.

Struktur yang digunakan untuk menyimpan simpul DOM Tree adalah sebagai berikut, simpul DOM Tree akan dibagi menjadi dua kasus, antara menjadi elemen HTML atau hanya teks biasa,

```
#[derive(Debug, Clone, PartialEq)]
pub enum Node {
    Element(Arc<Element>),
    Text(Arc<String>)
}

#[derive(Debug, Clone, PartialEq)]
pub struct Element {
    pub tag: String,
    pub attributes: HashMap<String, String>,
    pub children: Vec<Node>
}
```

Kemudian struktur berikut ini yang akan menjadi basis dari proses *parsing* CSS Selector, struktur tersebut dibuat berdasarkan spesifikasi tipe-tipe selector CSS,

```
#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
pub enum Combinator {
    Descendant,
    Child,
    DirectNextSibling,
    NextSibling,
}

#[derive(Debug, Clone, PartialEq, Serialize, Deserialize)]
pub enum Namespace {
    Default,
    Any,
    None,
    Named(String),
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct AttributeFilter {
    pub namespace: Namespace,
    pub name: String,
    pub operator: Option<String>,
    pub value: Option<String>,
}
```

```

    pub modifier: Option<char>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct PseudoFilter {
    pub name: String,
    pub args: Option<String>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct SelectorUnit {
    pub namespace: Namespace,
    pub tag: Option<String>,
    pub ids: Option<Vec<String>>,
    pub classes: Option<Vec<String>>,
    pub attributes: Option<Vec<AttributeFilter>>,
    pub pseudos: Option<Vec<PseudoFilter>>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct NodeFilter {
    pub prev_combinator: Option<Combinator>,
    pub selector: SelectorUnit,
}

#[derive(Clone)]
struct CssLexerWrapper<'a> {
    tokenizer: CssLexer<'a>,
    current_token: CssToken,
    current_text: &'a str,
    current_start_pos: usize,
}

#[derive(Clone)]
pub struct CssSelectorParser<'a> {
    css_lexer: CssLexerWrapper<'a>,
    is_relative_selector: bool,
    error_message: String,
    pub traversal: CssSelectorTraversal,
}

```

```
}
```

Lalu struktur yang mendeskripsikan perintah untuk dijalankan oleh thread turunan yaitu,

```
#[derive(Debug, Clone)]
pub struct ThreadTask {
    pub curr_node          : Arc<Element>,
    pub node_child_pos     : usize,
    pub parent_node       : Arc<Element>,
    pub selector_list_idx  : usize,
    pub selector_unit_idx  : usize,
    pub depth              : usize
}
```

Struktur tersebut dapat digunakan pada variasi struktur data global *thread safe* berikut,

```
#[derive(Debug, Clone)]
pub struct AsyncVec<T> {
    pub data: Arc<Mutex<Vec<T>>>,
}

#[derive(Debug, Clone)]
pub struct AsyncHashSet<T> {
    pub data: Arc<Mutex<HashSet<T>>>,
}

#[derive(Debug, Clone)]
pub struct AsyncStack<T> {
    pub data: Arc<Mutex<VecDeque<T>>>,
}

#[derive(Debug, Clone)]
pub struct AsyncQueue<T> {
    pub data: Arc<Mutex<VecDeque<T>>>,
}
```

Terakhir terdapat struktur berikut yang menjadi hasil dari *pre-processing* teknik *binary lifting*. Struktur tersebut berisi tabel prekomputasi *ancestor* dan juga HashTable untuk pemetaan sesuai yang dijelaskan pada 3.2,

```
pub struct BinaryLiftMetadata {
    ancestors          : Vec<Vec<usize>>,
    max_precompute_height : usize,
    depth_map          : Vec<usize>,
    id_to_node_map      : Vec<Node>,
    node_to_id_map      : HashMap<usize, usize>
}
```

Dengan setiap struktur tersebut, diimplementasikan logika utama yang dibutuhkan untuk melakukan traversal DFS dan BFS secara parallel, dan LCA secara non-parallel,

```
pub fn async_bfs(html_text: &str, css_query: &str, thread_num: usize)
    -> Option<Vec<Arc<Element>>>;

pub fn async_dfs(html_text: &str, css_query: &str, thread_num: usize)
    -> Option<Vec<Arc<Element>>>;

pub fn preprocess_tree(root: &Node) -> BinaryLiftMetadata;

pub fn find_lca(node1: &Node, node2: &Node, metadata: &BinaryLiftMetadata)
```

4.1.2 Frontend Application

Implementasi aplikasi frontend dilakukan menggunakan *library* **Yew**, yang digabungkan dengan *framework* **Tailwind CSS**.

Detail implementasi meliputi beberapa file berikut:

- **bindings.rs**, file ini berisi *binding* untuk fungsi terkait Web Assembly.
- **canvas_tree.rs**, file ini berisi logika penggambaran DOM Tree dalam antarmuka pengguna.
- **html_input.rs**, file ini berisi *handler* masukan HTML, baik melalui file ataupun scraping.
- **left_section.rs**, file ini berisi *handler* untuk bagian panel antarmuka di bagian kiri.
- **right_section.rs**, file ini berisi *handler* untuk bagian panel antarmuka di bagian kanan.
- **js_util.rs**, file ini berisi fungsi JavaScript pembantu tampilan.

- **result_player.rs**, file ini berisi logika penayangan animasi yang berkaitan dengan traversal.
- **main.rs**, file ini berisi pemanggilan fungsi utama dari *framework frontend Yew*.

Aplikasi frontend akan mengirim input melalui HTTP Request ke server, kemudian server akan memproses request tersebut. Jika server menerima parameter URL maka akan dianggap sebagai perintah untuk melakukan *web scraping*.

```
#[derive(Serialize)]
struct QueryPostBody {
    input_type: String,
    css_query: String,
    file_payload: String,
    url_payload: String,
    text_payload: String,
    use_dfs: bool,
}
```

Kemudian hasil dari *web scraping* akan dikirim ke *handler* CSS Selector, dan dilanjutkan ke traversal utama. Hasil dari traversal tersebut disimpan dalam sebuah struktur yang menyimpan setiap paths sebagai path dari akar lokal ke setiap simpul yg ditemukan,

```
#[derive(Clone, PartialEq)]
pub struct ResultItem {
    pub query: String,
    pub paths: Vec<Vec<u32>>,
    pub selected: Vec<u32>,
    pub traversal_path: Vec<u32>,
}
```

Atribut *selected* memiliki arti index simpul yg ditemukan, dan *traversal path* adalah gabungan dari semua path yg dilalui pada suatu step dimana ketika ada index simpul yang sama artinya terjadi *backtracking*.

4.2 Pengujian

5 Kesimpulan dan Saran

5.1 Kesimpulan

Sangat banyak permasalahan nyata yang dapat dimodelkan melalui graf. Sehingga algoritma graf sangat berguna untuk pemecahan masalah secara umum, terutama algoritma sederhana seperti traversal melalui DFS/BFS.

Terlihat bahwa permasalahan mencocokkan CSS Selector pada HTML DOM Tree yang merupakan tugas kompleks bagi Web Browser moderen dapat dimodelkan dengan graf khusus berbentuk tree, dan dapat diselesaikan melalui BFS/DFS.

5.2 Saran

Permasalahan terkait Web Browser moderen seharusnya lebih digali dan dikembangkan lagi dalam ilmu keinformatikaan, hal ini karena hampir semua teknologi komputasi sudah berhubungan langsung dengan Web, dan juga berbagai sub-permasalahan dalam pembuatan Web Browser sangat berguna untuk melatih ilmu informatika.

Pernyataan Orisinilitas

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Muhammad Fatih Irkham Mauludi (13524004)



Josh Reinhart Zidik (13524048)



Jingglang Galih Rinenggan (13524095)

Lampiran

Repository Github:

https://github.com/FieryBanana101/Tubes2_kata-josh-nama-timnya-jangan-ini

Table 1: Checklist hasil

No	Poin	Ya	Tidak
1.	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2.	Aplikasi berhasil dijalankan	✓	
3.	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4.	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5.	Aplikasi dapat menampilkan visualisasi pohon DOM		✓
6.	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran		✓
7.	Aplikasi dapat menandai jalur tempuh oleh algoritma		✓
8.	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log		✓
9.	[Bonus] Membuat video		✓
10.	[Bonus] Deploy aplikasi		✓
11.	[Bonus] Implementasi animasi pada penelusuran pohon		✓
12.	[Bonus] Implementasi multithreading	✓	
13.	[Bonus] Implementasi LCA Binary Lifting	✓	

Table 2: Pembagian Tugas Kelompok

No	Anggota	Detail Tugas	Presentase Keseluruhan
1.	Muhammad Fatih Irkham Mauludi (13524004)	Keseluruhan CSS Selector Parser, DFS + BFS dengan <i>multi-threading</i> , LCA + Binary Lifting, testing keseluruhan, dan pembuatan keseluruhan laporan	33.3333%
2.	Josh Reinhart Zidik (13524048)	Keseluruhan HTML Parser, testing keseluruhan	33.3333%
3.	Jingglang Galih Rinenggan (13524095)	Keseluruhan aplikasi frontend, animasi, web scraping, deployment, testing keseluruhan	33.3333%

Daftar Pustaka

- [1] Munir, Rinaldi. "Breadth/Depth First Search (BFS/DFS) (Bagian 1)" Strategi Algoritma, STEI-ITB, 2026, [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-\(2026\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-(2026)-Bagian1.pdf). Accessed 18 April 2026.
- [2] Munir, Rinaldi. "Breadth/Depth First Search (BFS/DFS) (Bagian 2)" Strategi Algoritma, STEI-ITB, 2026, [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/14-BFS-DFS-\(2026\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/14-BFS-DFS-(2026)-Bagian2.pdf). Accessed 18 April 2026.
- [3] jakobkogler, paramsingh, et al. "Depth First Search" Algorithms for Competitive Programming, cp-algorithms, 2025, <https://cp-algorithms.com/graph/depth-first-search.html>. Accessed 20 April 2026.
- [4] jakobkogler, paramsingh, et al. "Breadth-first search" Algorithms for Competitive Programming, cp-algorithms, 2024, <https://cp-algorithms.com/graph/breadth-first-search.html>. Accessed 20 April 2026.
- [5] jakobkogler, pin3da, et al. "Lowest Common Ancestor" Algorithms for Competitive Programming, cp-algorithms, 2025, <https://cp-algorithms.com/graph/lca.html>. Accessed 20 April 2026.
- [6] jakobkogler, mhayter, adamant-pwn. "Solve RMQ (Range Minimum Query) by finding LCA (Lowest Common Ancestor)" Algorithms for Competitive Programming, cp-algorithms, 2025, https://cp-algorithms.com/graph/rmq_linear.html. Accessed 20 April 2026.
- [7] jakobkogler, adamant-pwn, et al. "Lowest Common Ancestor - Binary Lifting" Algorithms for Competitive Programming, cp-algorithms, 2026, https://cp-algorithms.com/graph/lca_binary_lifting.html. Ac-

cessed 20 April 2026.

- [8] Laaksonen, Antti. "Competitive Programmer's Handbook", cses.fi, 2018, <https://cses.fi/book/book.pdf> Accessed 20 April 2026.
- [9] MDN contributors. "HTML: HyperText Markup Language" HTML, Mozilla.org, 2025, <https://developer.mozilla.org/en-US/docs/Web/HTML>. Accessed 25 April 2026.
- [10] MDN contributors. "Document Object Model" HTML, Mozilla.org, 2026, https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model. Accessed 25 April 2026.
- [11] HTML WG. "Parsing HTML Documents" HTML5, W3C.org, 2011, <https://www.w3.org/TR/2011/WD-html5-20110113/parsing.html>. Accessed 25 April 2026.
- [12] MDN contributors. "What is CSS?" Learn Web Development, Mozilla.org, 2025, https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics/What_is_CSS. Accessed 25 April 2026.
- [13] MDN contributors. "CSS selectors" CSS, Mozilla.org, 2026, <https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors>. Accessed 25 April 2026.
- [14] CSS WG. "Selectors Level 4" W3C Editor's Draft, W3C.org, 2026, <https://drafts.csswg.org/selectors-4/>. Accessed 25 April 2026.
- [15] Barney, Blaise. "Introduction to Parallel Computing Tutorial" Tutorials, Livermore Computing, 2021, <https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial>. Accessed 21 April 2026.
- [16] Silberschatz, Abraham, Peter B. Galvin, and Greg Gagne. Operating System Concepts. 9th ed., Wiley, 2012.

-
- [17] Reese, George. "Chapter 7: Distributed Application Architecture." Database Programming with JDBC Java, 2nd ed., O'Reilly Media, Inc., 2000, <https://web.archive.org/web/20110406121920/http://java.sun.com/developer/Books/jdbc/ch07.pdf>. Accessed 21 April 2026.
- [18] Nigel, Poulton. "Docker Deep Dive" Leanpub. 2018, <https://ebooks.karbust.me/Technology/Docker%20Deep%20Dive%20-%20Nigel%20Poulton.pdf>. Accessed 21 April 2026.
- [19] Martin, Enrico. "Virtualization and Containerization". 2021, Final Thesis, 2021, <https://unitesi.unive.it/retrieve/cc1c017f-1a66-443d-b375-a0adc1e0fba1/815635-84509.pdf>. Accessed 21 April 2026.